

任意对象查找器

文档 v1.0

首先, 感谢你通过 Unity 资源商店购买本插件。本文档将引导你完成使用流程, 以确保该工具能够在你的 Unity 项目中正常工作。

任意对象查找器 是什么 ?	1
功能特性.....	2
1. Search Methods.....	2
2. Targets (可多选):.....	4
3. Type.....	4
4. Location.....	4
5. Traversal Mode.....	5
6. Field Name.....	5
自定义.....	5
技术支持.....	7

任意对象查找器 是什么 ?

任意对象查找器是一款编辑器工具, 用于在场景和或项目文件夹中搜索任意 Unity 对象, 包括组件、预制体、资源, 以及所有派生自 `UnityEngine.Object` 的类型。项目文件夹范围同时包含 Assets 与 Packages。

工具会根据字段或成员的值条件进行递归匹配。同时支持在数组以及可序列化类实例中继续向下搜索。如果项目中使用了 Odin Serializer, 也可以对字典类型进行搜索。

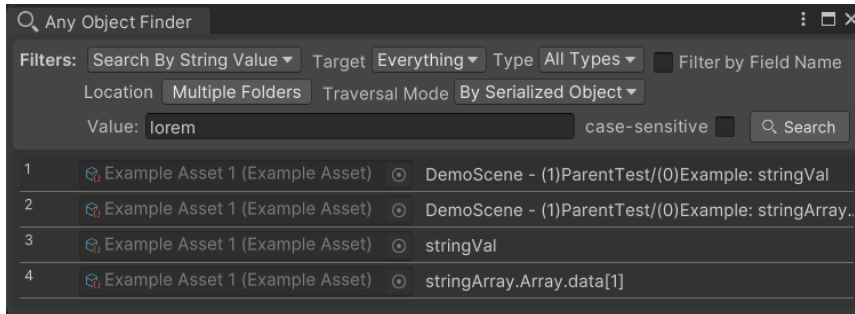
支持的匹配类型包括: 字符串、整数、浮点数、布尔值、枚举、Unity 对象引用, 以及可通过脚本扩展的自定义条件。你还可以使用高级过滤器来进一步缩小搜索范围。

搜索结果将以列表形式展示, 其中包含对象引用以及对应的字段名称或路径。对于场景对象或预制体对象, 还会额外显示其在层级中的 `GameObject` 路径。

功能特性

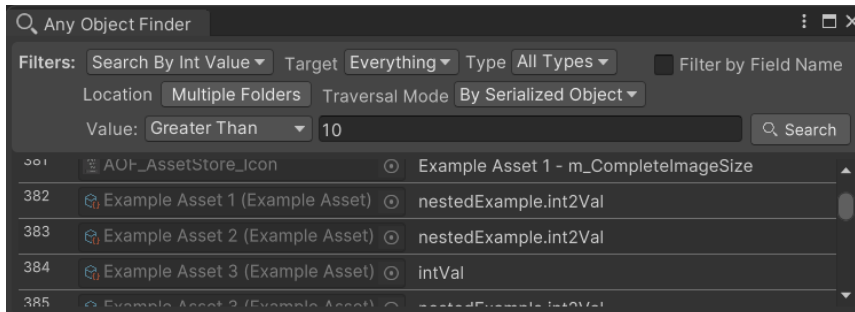
1. Search Methods

a. Search by String Value



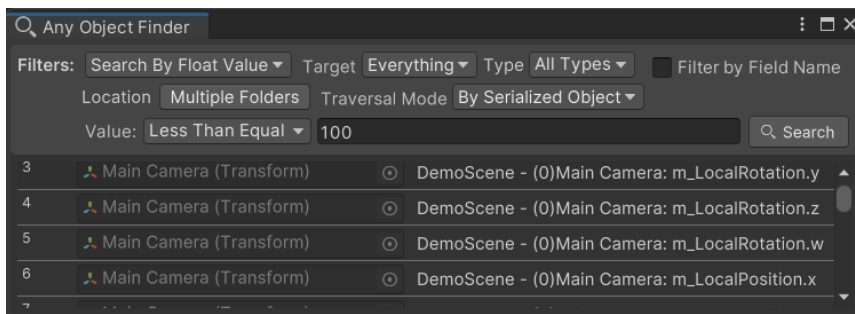
在所有经过筛选的对象中，查找其序列化属性里类型为字符串且与输入文本匹配的内容。参数为一个字符串值，并可指定是否区分大小写。

b. Search by Int Value



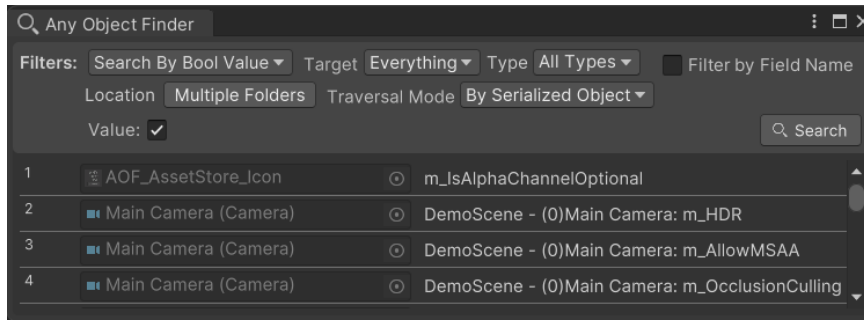
在所有经过筛选的对象中，查找其序列化属性里类型为整数且满足指定条件的字段。参数包括条件类型 (Equal、Less Than、Less Than Equal、More Than、More Than Equal、Not Equal) 以及用于比较的整数值。

c. Search by Float Value



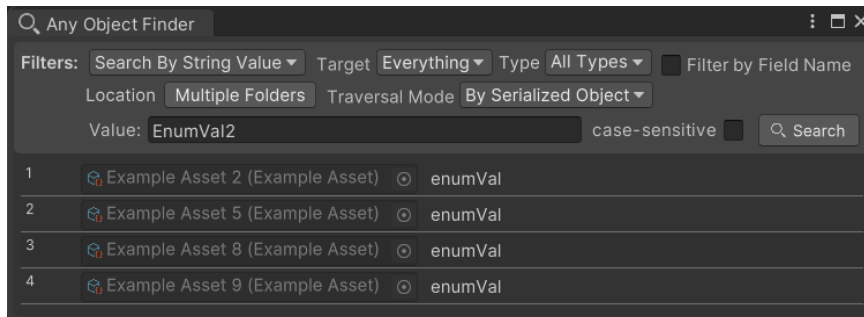
在所有经过筛选的对象中，查找其序列化属性里类型为浮点数且满足指定条件的字段。参数包括条件类型 (Nearly Equal、Less Than、Less Than Equal、More Than、More Than Equal、Not Equal) 以及用于比较的浮点数值。

d. Search by Boolean Value



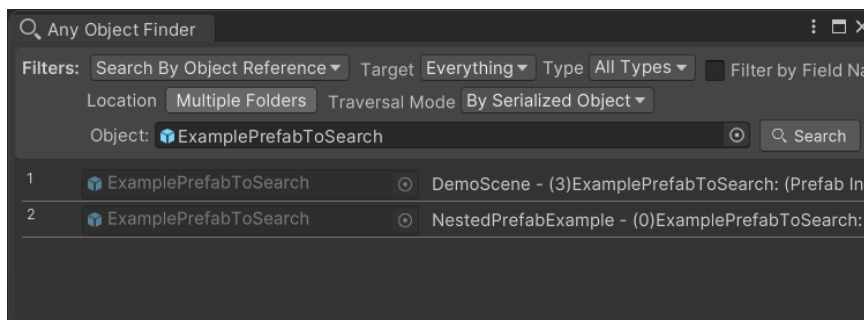
在所有经过筛选的对象中，查找其序列化属性里类型为布尔值且与指定 Value 相同的字段。

e. Search by Enum Value

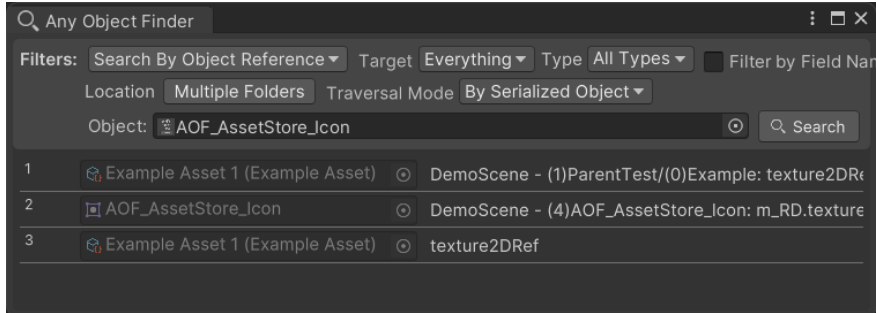


在所有经过筛选的对象中，查找其序列化属性里类型为枚举且其值名称与输入文本匹配的字段。参数为一个字符串值，并可指定是否区分大小写。

f. Search by Object Reference



在所有经过筛选的对象中，查找其序列化属性或预制体实例中类型为 Unity 对象且与指定对象引用相同的字段。



如果指定的对象是一个非预制体的资源，则会搜索所有引用该对象的其他对象。

g. Custom Methods (extend by script)

你可以通过扩展脚本来添加更多自定义搜索方式，甚至包括自定义参数的 GUI。相关内容将在“自定义”章节中介绍。

2. Targets (可多选):

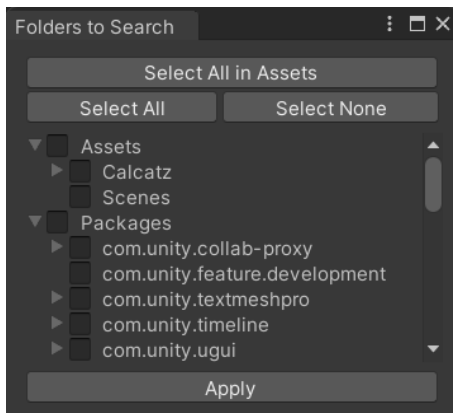
- **Scenes**
 - 在场景中进行搜索。搜索范围为指定文件夹位置下的所有场景资源。
- **Assets**
 - 在指定的文件夹位置中搜索资源。

3. Type

- 所有类型
- 或选择一个派生自 `UnityEngine.Object` 的指定类。

4. Location

选择要搜索的文件夹，范围同时包括 Assets 和 Packages。



5. Traversal Mode

用于在遍历所有字段时选择的遍历方式。基本思路是需要检查每一个 public 字段，以及所有带有 *SerializeField* 特性的字段。

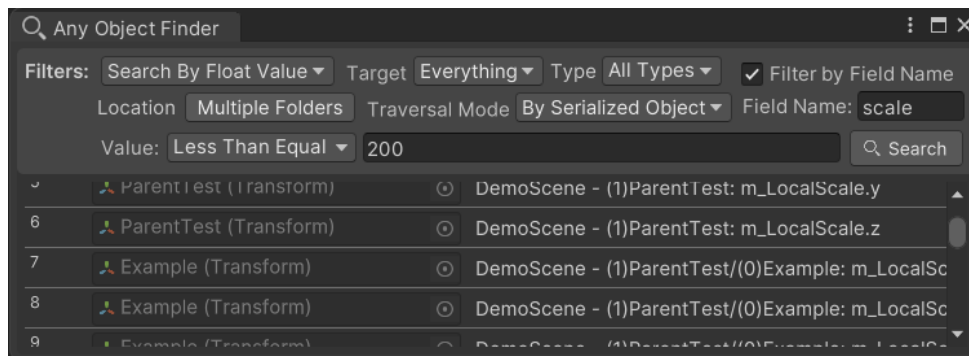
- **By Serialized Object**

通过遍历所有 *SerializedProperty*，使用 Unity 的 *SerializedObject* 来访问全部字段。这是默认推荐的遍历模式，因为它可以轻松找到 Unity 的内置字段。这些字段大多不是通过 C# 特性进行序列化，而是来自原生代码。

- **By Reflection**

使用 C# 反射来遍历所有字段。目前不太推荐这种方式，因为在搜索序列化字段方面能力较弱，无法检测到以原生方式序列化的内置字段。不过在未来版本中，可以基于此扩展出对未序列化字段的搜索能力，这在某些运行时扫描场景下可能会有帮助。

6. Field Name



通常，在搜索基础类型字段时，尤其是字符串以外的类型，得到的结果往往会比预期多。为了缩小范围，可以勾选 **Filter by Field Name**，并在 **Field Name** 中填入你希望限定搜索的字段名称。

自定义

可以通过创建一个继承自 *BaseSearchBy* 的新类来添加自定义搜索方式。

下面是可重写的方法列表：

1. *FilterObject*
 - 过滤逻辑。
2. *CreateGUI*
 - 当需要额外的参数字段时，用于绘制对应的 GUI 区域。
3. *OnCreateResultItemGUI*
 - 每个搜索结果项的 GUI 模板。通常做法是在左侧绘制对象字段，在右侧显示字段路径的文本。

4. OnBindResultItemGUI

- 将实际的结果数据填充到之前创建好的结果项 GUI 中。

你可以参考示例脚本 `MyCustomSearch`。该示例用于查找包含奇数整数值字段的对象。



以下是该示例的完整脚本：

```
#if UNITY_EDITOR

using Calcatz.AnyObjectFinder;
using System.Collections.Generic;
using UnityEditor;
using UnityEngine;
using UnityEngine.UIElements;

/// <summary>
/// A custom search-by example where it searches fields with odd integer value.
/// </summary>
public class MyCustomSearch : BaseSearchBy {

    // The filter logic
    public override List<ObjectFinder.ResultItem> FilterObject(ObjectFinder.TraversalMode traversalMode,
                                                                string fieldNameFilter,
                                                                Object rootObject,
                                                                Object objectToSearch,
                                                                SerializableSearchArguments searchArguments) {

        var resultItems = new List<ObjectFinder.ResultItem>();

        ComparerMethod<int> comparer = fieldValue => {
            return fieldValue % 2 == 1;
        };

        FieldFoundCallback<int> onFieldFound = (objectReference, fieldValue, fieldPath) => {

            if (rootObject != objectToSearch) {
                if (rootObject is SceneAsset && objectToSearch is Component component) {
                    fieldPath = GetSceneObjectPath(component) + ":" + fieldPath;
                }
            }

            if (resultItems == null) resultItems = new List<ObjectFinder.ResultItem>();
            resultItems.Add(new ObjectFinder.ResultItem() {
                targetObject = objectReference,
                info = rootObject == objectReference ? fieldPath : rootObject.name + " - " + fieldPath
            });
        };

        if (traversalMode.HasFlag(ObjectFinder.TraversalMode.BySerializedObject)) {
```

```
        TraverseSerializedObject<int>(fieldNameFilter, comparer, onFieldFound, objectToSearch);
    }
    if (traversalMode.HasFlag(ObjectFinder.TraversalMode.ByReflection)) {
        TraverseFields<int>(fieldNameFilter, comparer, onFieldFound, objectToSearch);
    }
    return resultItems;
}

// GUI area if additional parameter fields are needed.
public override void CreateGUI(VisualElement customAreaRoot, SerializedProperty serializedProperty) {
    base.CreateGUI(customAreaRoot, serializedProperty);
    customAreaRoot.Add(new Label("If additional parameters are needed, use this area.));
}

// GUI template for each search result item.
public override void OnCreateResultItemGUI(VisualElement parentElement) {
    base.OnCreateResultItemGUI(parentElement);
    parentElement.Add(CreateSelectableTextElement());
}

// Fill the contents of the previously created result item GUI with the actual result item data.
public override void OnBindResultItemGUI(VisualElement parentElement, int index, ObjectFinder.ResultItem resultItem) {
    base.OnBindResultItemGUI(parentElement, index, resultItem);
    if (parentElement.ElementAt(2) is TextField textField) {
        textField.value = resultItem.info;
    }
}

}

#endif
```

技术支持

如果你发现了任何问题, 或对本工具有相关建议, 可以发送邮件至 affan@calcatz.com, 或在 Twitter 上通过 [@dilaura_exp](https://twitter.com/dilaura_exp) 与我联系。